

Kubernetes Day 6 3 June 2026

```
Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D (main)
```

```
$ eksctl create cluster --name prod --nodegroup-name b17dng --node-type t3.micro --nodes 5 --managed
```

```
Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D (main)
```

```
$ eksctl get cluster
```

NAME	REGION	EKSCTL	CREATED
prod	us-east-1	True	

```
Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D (main)
```

```
$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
ip-192-168-17-129.ec2.internal	Ready	<none>	4m57s	v1.34.8-eks-3385e9b
ip-192-168-21-182.ec2.internal	Ready	<none>	4m57s	v1.34.8-eks-3385e9b
ip-192-168-31-61.ec2.internal	Ready	<none>	4m57s	v1.34.8-eks-3385e9b
ip-192-168-50-167.ec2.internal	Ready	<none>	4m52s	v1.34.8-eks-3385e9b
ip-192-168-53-4.ec2.internal	Ready	<none>	4m54s	v1.34.8-eks-3385e9b

```
Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D (main)
```

```
$ cd Kubernetes/ practice/3_June/
```

```
Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/3_June (main)
```

```
$ ll
```

```
total 4
```

```
-rw-r--r-- 1 Bhavana 197121 656 Jun  4 05:38 2_instadeploy_v1.yaml  
-rw-r--r-- 1 Bhavana 197121   0 Jun  4 05:38 3_instadeploy_v2.yaml
```

```
#2_instadeploy_v1.yaml
```

```
---  
apiVersion: apps/v1  
kind: Deployment  
metadata:  
  name: fb-deployment #name should be same in other (v2) version of deployment yaml  
  file  
  labels:  
    app: fb-deploy  
    env: prod  
spec:  
  replicas: 3  
  strategy:  
    type: RollingUpdate  
    rollingUpdate:  
      maxUnavailable: 1  
      maxSurge: 1  
  selector:  
    matchLabels:  
      app: fb-app  
      env: prod  
  template:  
    metadata:  
      labels:  
        app: fb-app  
        env: prod
```

```

    version: v1
spec:
  containers:
  - name: fb-container
    image: dockerhub1105/practicerepo:mc_app_img
    ports:
    - containerPort: 8080

```

Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/3_June (main)

```
$ kubectl apply -f 2_instadeploy_v1.yaml
deployment.apps/fb-deployment created
```

Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/3_June (main)

```
$ kubectl get all
```

NAME	READY	STATUS	RESTARTS	AGE
pod/fb-deployment-5c47c87456-gtd42	1/1	Running	0	8s
pod/fb-deployment-5c47c87456-sgbmr	1/1	Running	0	8s
pod/fb-deployment-5c47c87456-xv5gd	1/1	Running	0	8s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/kubernetes	ClusterIP	10.100.0.1	<none>	443/TCP	11m

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/fb-deployment	3/3	3	3	9s

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/fb-deployment-5c47c87456	3	3	3	10s

Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/3_June (main)

```
$ kubectl describe deploy fb-deployment
```

```

Name:                fb-deployment
Namespace:           default
CreationTimestamp:    Thu, 04 Jun 2026 05:47:52 +0530
Labels:              app=fb-deploy
                    env=prod
Annotations:         deployment.kubernetes.io/revision: 1
Selector:             app=fb-app,env=prod
Replicas:            3 desired | 3 updated | 3 total | 3 available | 0
unavailable
StrategyType:        RollingUpdate
MinReadySeconds:     0
RollingUpdateStrategy: 1 max unavailable, 1 max surge
Pod Template:
  Labels:  app=fb-app
          env=prod
          version=v1
  Containers:
    fb-container:
      Image:      dockerhub1105/practicerepo:mc_app_img
      Port:       8080/TCP
      Host Port:  0/TCP
      Environment: <none>
      Mounts:      <none>
  Volumes:        <none>
  Node-Selectors:  <none>
  Tolerations:    <none>

```

```

Conditions:
  Type      Status  Reason
  ----      -
  Available True    MinimumReplicasAvailable
  Progressing True    NewReplicaSetAvailable
OldReplicaSets: <none>
NewReplicaSet:  fb-deployment-5c47c87456 (3/3 replicas created)

```

Events:

Type	Reason	Age	From	Message
Normal	ScalingReplicaSet	64s	deployment-controller	Scaled up replica set fb-deployment-5c47c87456 from 0 to 3

Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/3_June (main)

\$ kubectl get po -o wide

NAME	READY	STATUS	RESTARTS	AGE	IP
NODE	NOMINATED	NODE	READINESS	GATES	
fb-deployment-5c47c87456-gtd42	1/1	Running	0	3m59s	
192.168.10.111	ip-192-168-17-129.ec2.internal	<none>			<none>
fb-deployment-5c47c87456-sgbmr	1/1	Running	0	3m59s	
192.168.40.46	ip-192-168-53-4.ec2.internal	<none>			<none>
fb-deployment-5c47c87456-xv5gd	1/1	Running	0	3m59s	
192.168.49.69	ip-192-168-50-167.ec2.internal	<none>			<none>

Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/3_June (main)

\$ kubectl exec -it fb-deployment-5c47c87456-gtd42 -- bash

root@fb-deployment-5c47c87456-gtd42:/usr/local/tomcat/webapps# ll

```
total 32
drwxr-xr-x. 1 root root    18 Jun  4 00:18 ./
drwxr-xr-x. 1 root root    57 May  5 23:13 ../
drwxr-x---. 4 root root    54 Jun  4 00:18 ROOT/
-rw-r--r--. 1 root root 30164 May 13 09:19 ROOT.war
```

Exit

```
#4_lb-svc.yaml
```

```
---
apiVersion: v1
kind: Service
metadata:
  name: fb-service
  labels:
    app: fb-service
    env: prod
spec:
  type: LoadBalancer
  selector:
    app: fb-app
    env: prod
  ports:
    - protocol: TCP
      port: 80 # This is the load balancer port, which is a external port.
      targetPort: 8080 # This is the container port on which the application is running.
```

Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/3_June (main)

\$ kubectl apply -f 4_lb-svc.yaml

service/fb-service created

Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/3_June (main)

\$ kubectl get all

NAME	READY	STATUS	RESTARTS	AGE
pod/fb-deployment-5c47c87456-gtd42	1/1	Running	0	11m
pod/fb-deployment-5c47c87456-sgbmr	1/1	Running	0	11m
pod/fb-deployment-5c47c87456-xv5gd	1/1	Running	0	11m

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP
service/fb-service	LoadBalancer	10.100.183.47	
80:31672/TCP			a288533cb3d0148ea9fe08d9dfc0b286-2042922771.us-east-1.elb.amazonaws.com
service/kubernetes	ClusterIP	10.100.0.1	<none>
443/TCP			

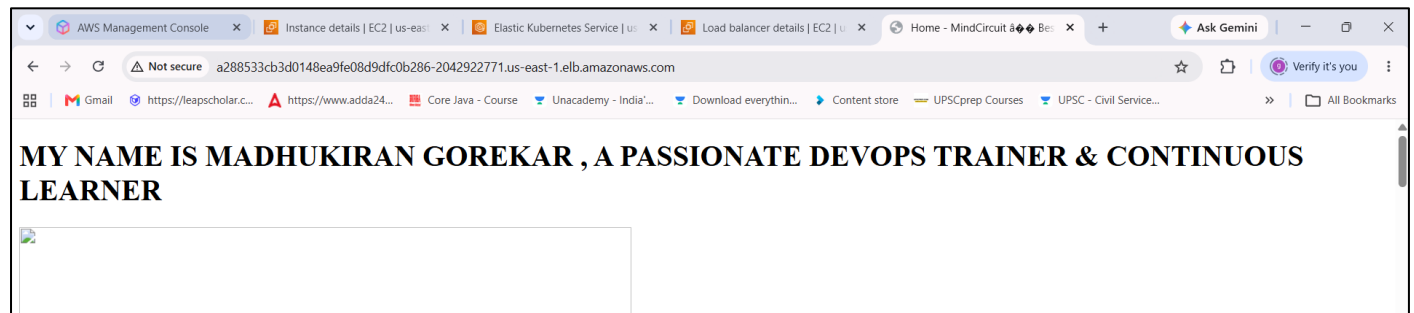
NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/fb-deployment	3/3	3	3	11m

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/fb-deployment-5c47c87456	3	3	3	11m

Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/3_June (main)

\$ kubectl get svc

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP
fb-service	LoadBalancer	10.100.183.47	a288533cb3d0148ea9fe08d9dfc0b286-2042922771.us-east-1.elb.amazonaws.com
80:31672/TCP			20s
kubernetes	ClusterIP	10.100.0.1	<none>
443/TCP			



```
#3_instadeploy_v2.yaml
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: fb-deployment #name should be same in other (v1) version of deployment yaml
file
  labels:
    app: fb-deploy
    env: prod
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1
      maxSurge: 1
  selector:
    matchLabels:
      app: fb-app
      env: prod
  template:
    metadata:
      labels:
```

```

    app: fb-app
    env: prod
    version: v2
spec:
  containers:
  - name: fb-container
    image: dockerhub1105/practicerepo:python_image_v1
    ports:
    - containerPort: 8080

```

Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/3_June (main)
\$ kubectl apply -f 3_instadeploy_v2.yaml
deployment.apps/fb-deployment configured

Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/3_June (main)
\$ kubectl get all

NAME	READY	STATUS	RESTARTS	AGE
pod/fb-deployment-7f8bb96d5c-58qbz	1/1	Running	0	15s
pod/fb-deployment-7f8bb96d5c-pbd44	1/1	Running	0	15s
pod/fb-deployment-7f8bb96d5c-shvr5	1/1	Running	0	14s

NAME	AGE	TYPE	CLUSTER-IP	EXTERNAL-IP
service/fb-service	6m15s	LoadBalancer	10.100.183.47	a288533cb3d0148ea9fe08d9dfc0b286-2042922771.us-east-1.elb.amazonaws.com
service/kubernetes	29m	ClusterIP	10.100.0.1	<none>

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/fb-deployment	3/3	3	3	17m

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/fb-deployment-5c47c87456	0	0	0	17m
replicaset.apps/fb-deployment-7f8bb96d5c	3	3	3	16s

Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/3_June (main)

```

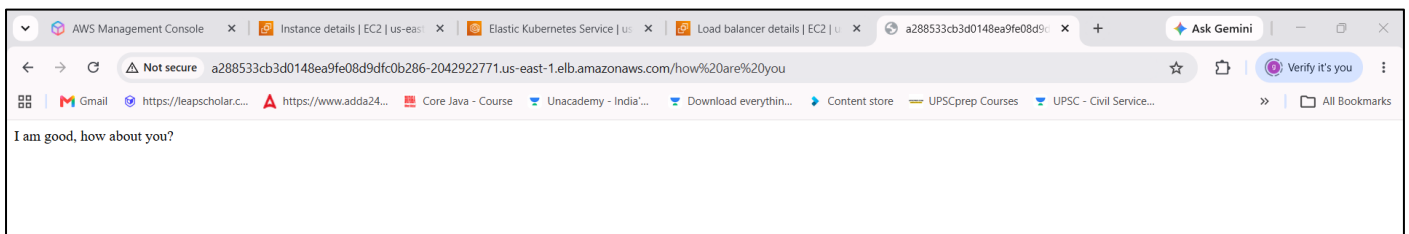
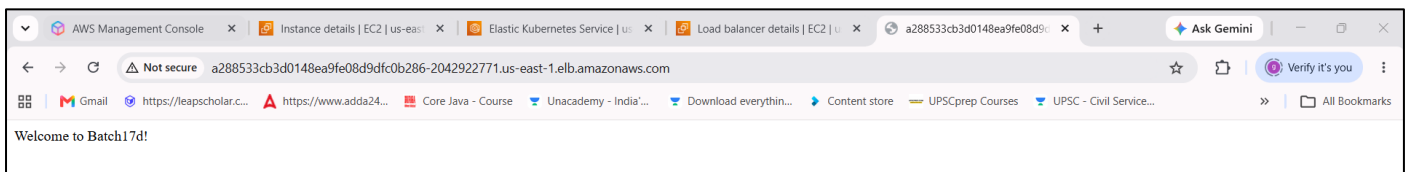
$ kubectl describe deploy fb-deployment
Name:                fb-deployment
Namespace:           default
CreationTimestamp:    Thu, 04 Jun 2026 05:47:52 +0530
Labels:              app=fb-deploy
                    env=prod
Annotations:         deployment.kubernetes.io/revision: 2
Selector:             app=fb-app,env=prod
Replicas:            3 desired | 3 updated | 3 total | 3 available | 0
unavailable
StrategyType:        RollingUpdate
MinReadySeconds:     0
RollingUpdateStrategy: 1 max unavailable, 1 max surge
Pod Template:
  Labels:  app=fb-app
          env=prod
          version=v2
  Containers:
    fb-container:
      Image:      dockerhub1105/practicerepo:python_image_v1
      Port:      8080/TCP
      Host Port: 0/TCP
      Environment: <none>
      Mounts:     <none>

```

```

volumes:          <none>
node-selectors:   <none>
tolerations:      <none>
conditions:
  Type              Status  Reason
  ----              -
  Available         True    MinimumReplicasAvailable
  Progressing       True    NewReplicaSetAvailable
oldReplicaSets:    fb-deployment-5c47c87456 (0/0 replicas created)
NewReplicaSet:     fb-deployment-7f8bb96d5c (3/3 replicas created)
Events:
  Type              Reason              Age   From                      Message
  ----              -
  Normal            ScalingReplicaSet   18m   deployment-controller     Scaled up replica set fb-deployment-5c47c87456 from 0 to 3
  Normal            ScalingReplicaSet   58s   deployment-controller     Scaled up replica set fb-deployment-7f8bb96d5c from 0 to 1
  Normal            ScalingReplicaSet   58s   deployment-controller     Scaled down replica set fb-deployment-5c47c87456 from 3 to 2
  Normal            ScalingReplicaSet   58s   deployment-controller     Scaled up replica set fb-deployment-7f8bb96d5c from 1 to 2
  Normal            ScalingReplicaSet   57s   deployment-controller     Scaled down replica set fb-deployment-5c47c87456 from 2 to 1
  Normal            ScalingReplicaSet   57s   deployment-controller     Scaled up replica set fb-deployment-7f8bb96d5c from 2 to 3
  Normal            ScalingReplicaSet   57s   deployment-controller     Scaled down replica set fb-deployment-5c47c87456 from 1 to 0

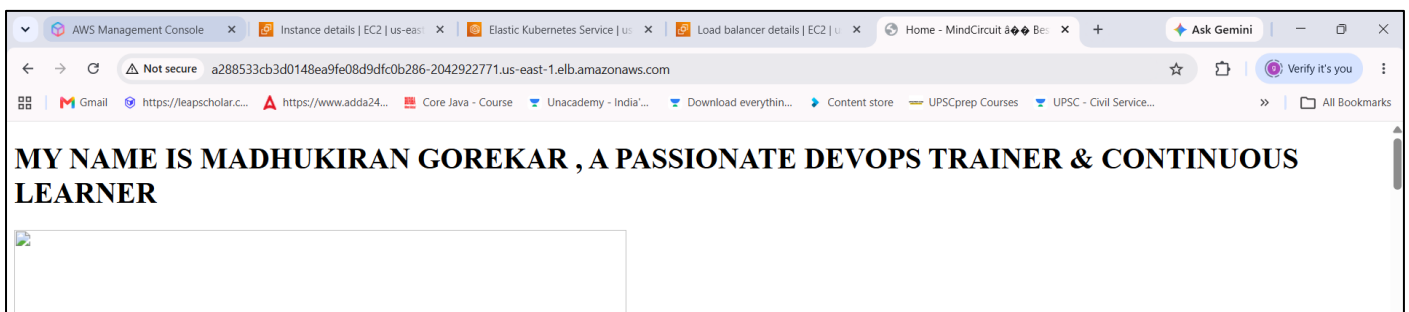
```



```

Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-
Devops_MC_B17D/Kubernetes/practice/3_June (main)
$ kubectl rollout undo deploy fb-deployment

```



```

Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-
Devops_MC_B17D/Kubernetes/practice/3_June (main)
$ kubectl delete deploy fb-deployment
deployment.apps "fb-deployment" deleted from default namespace

```

```

Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-
Devops_MC_B17D/Kubernetes/practice/3_June (main)
$ kubectl get all

```


NAME	TYPE	CLUSTER-IP	EXTERNAL-IP
service/fb-service	LoadBalancer	10.100.183.47	
a288533cb3d0148ea9fe08d9dfc0b286-2042922771.us-east-1.elb.amazonaws.com			
80:31672/TCP			
17m			
service/kubernetes	ClusterIP	10.100.0.1	<none>
443/TCP			
40m			

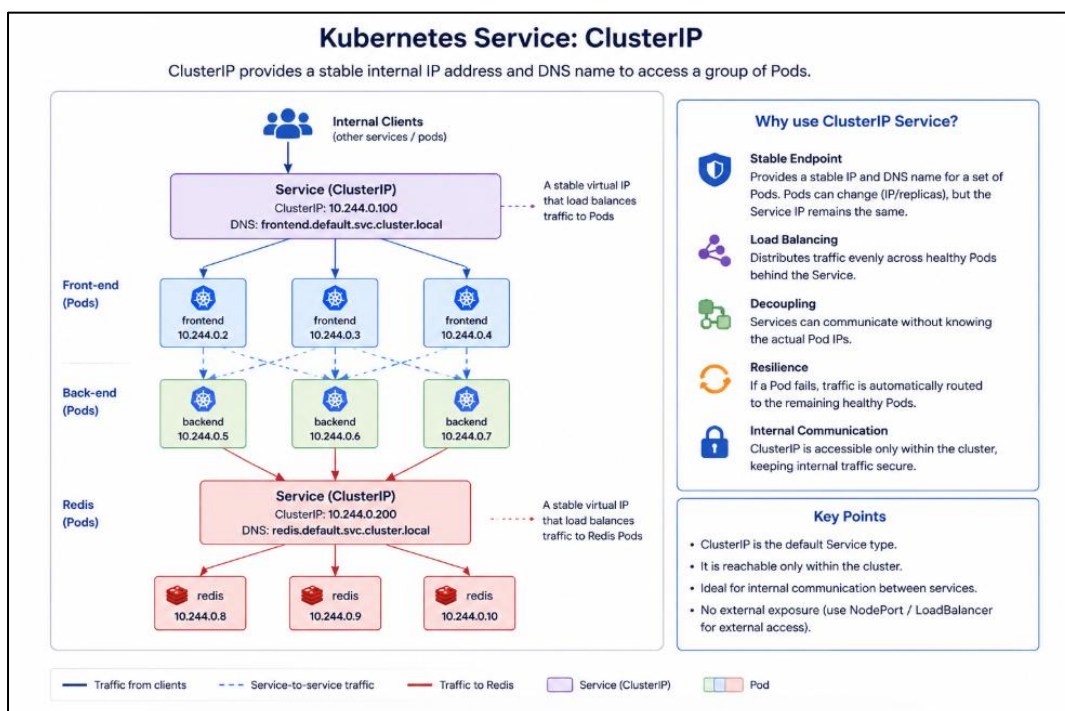
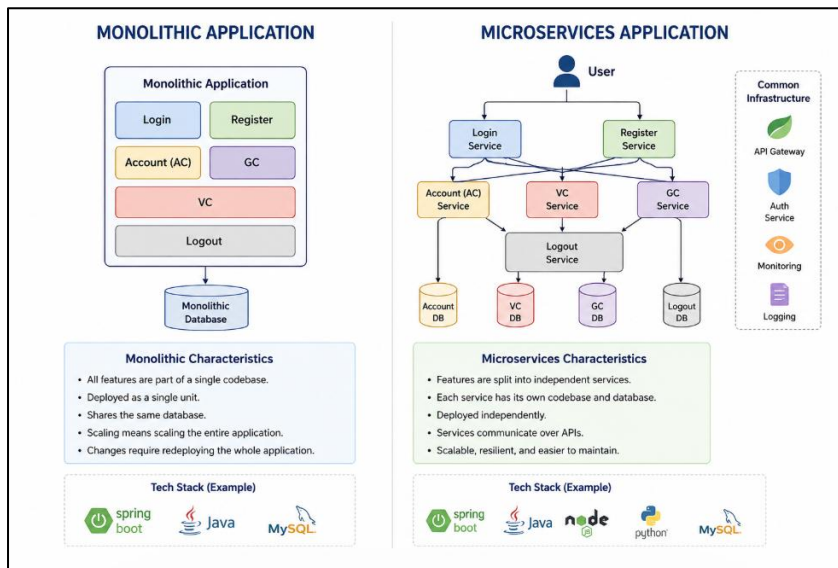
Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/3_June (main)

\$ kubectl delete svc fb-service
service "fb-service" deleted from default namespace

Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/3_June (main)

\$ kubectl get all

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/kubernetes	ClusterIP	10.100.0.1	<none>	443/TCP	41m



```
#5_clusterip_svc.yaml
---
apiVersion: v1
kind: Service
metadata:
  name: fb-service
  labels:
    app: fb-service
    env: prod
spec:
  type: ClusterIP #we use clusterip when we want to access the service only within the
cluster.
  selector:
    app: fb-app
    env: prod
  ports:
    - protocol: TCP
      port: 80 # This is the port to access the service, which is a internal port.
      targetPort: 8080 # This is the port on which the application is running inside
the container.
```

Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/3_June (main)
\$ kubectl apply -f 5_clusterip-svc.yaml
service/fb-service configured

Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/3_June (main)
\$ kubectl get all

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/fb-service	ClusterIP	10.100.59.42	<none>	80/TCP	27s
service/kubernetes	ClusterIP	10.100.0.1	<none>	443/TCP	60m

Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/3_June (main)
\$ kubectl get svc

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
fb-service	ClusterIP	10.100.59.42	<none>	80/TCP	50s
kubernetes	ClusterIP	10.100.0.1	<none>	443/TCP	60m

Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/3_June (main)
\$ kubectl apply -f 3_instadeploy_v2.yaml
deployment.apps/fb-deployment created

Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/3_June (main)
\$ kubectl get all

NAME	READY	STATUS	RESTARTS	AGE
pod/fb-deployment-58d8bd65b9-n5bzg	1/1	Running	0	9s
pod/fb-deployment-58d8bd65b9-rdrj6	1/1	Running	0	9s
pod/fb-deployment-58d8bd65b9-znrsj	1/1	Running	0	9s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/fb-service	ClusterIP	10.100.59.42	<none>	80/TCP	114s
service/kubernetes	ClusterIP	10.100.0.1	<none>	443/TCP	61m

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/fb-deployment	3/3	3	3	10s

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/fb-deployment-58d8bd65b9	3	3	3	10s

```
Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/3_June (main)
$ kubectl apply -f 4_lb-svc.yaml
service/fb-service configured
```

```
Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/3_June (main)
$ kubectl get all
```

NAME	READY	STATUS	RESTARTS	AGE
pod/fb-deployment-58d8bd65b9-n5bzg	1/1	Running	0	32s
pod/fb-deployment-58d8bd65b9-rdrj6	1/1	Running	0	32s
pod/fb-deployment-58d8bd65b9-znrsj	1/1	Running	0	32s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP
service/fb-service	LoadBalancer	10.100.59.42	
ab66a2684e4df455cbdd571b4d131320-1186501111.us-east-1.elb.amazonaws.com			
80:30151/TCP			
service/kubernetes	ClusterIP	10.100.0.1	<none>
443/TCP			

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/fb-deployment	3/3	3	3	34s

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/fb-deployment-58d8bd65b9	3	3	3	34s

```
Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/3_June (main)
```

```
$ kubectl get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP
fb-service	LoadBalancer	10.100.59.42	ab66a2684e4df455cbdd571b4d131320-1186501111.us-east-1.elb.amazonaws.com
80:30151/TCP			2m31s
kubernetes	ClusterIP	10.100.0.1	<none>
443/TCP			62m

```
Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/3_June (main)
```

```
$ kubectl get po -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP
fb-deployment-58d8bd65b9-n5bzg	1/1	Running	0	65s	
192.168.10.111		ip-192-168-17-129.ec2.internal	<none>		<none>
fb-deployment-58d8bd65b9-rdrj6	1/1	Running	0	65s	
192.168.49.69		ip-192-168-50-167.ec2.internal	<none>		<none>
fb-deployment-58d8bd65b9-znrsj	1/1	Running	0	65s	
192.168.40.46		ip-192-168-53-4.ec2.internal	<none>		<none>

```
Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/3_June (main)
```

```
$ kubectl exec -it fb-deployment-58d8bd65b9-n5bzg -- bash
```

```
bash-5.2# ls
bin boot dev etc home lib lib64 local media mnt opt proc root run
sbin srv sys tmp usr var
bash-5.2# curl 10.100.0.1:80
```

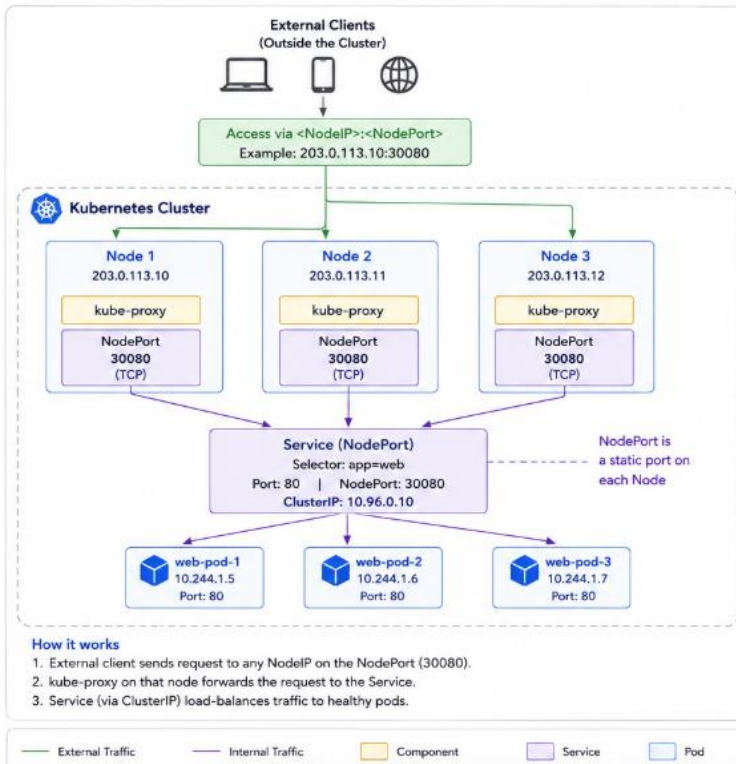
```
----- you can communicate to other service (i.e internal comm'n use cluterIp service)
```

```
^C
bash-5.2# exit
```

```
exit
command terminated with exit code 130
```

Kubernetes Service: NodePort

NodePort exposes a Service on a static port on each Node's IP, making the Service accessible from outside the cluster.



Why use NodePort Service?



External Access

Exposes the Service outside the cluster using <NodeIP>:<NodePort>.



Simple & Easy

Easy to set up and understand.



No Extra Components

Does not require external LoadBalancer.



Static Port

NodePort is a static port (30000-32767) on each node.

Key Points

- NodePort exposes the Service on each Node's IP at a static port.
- Port range: 30000-32767 (default).
- Accessible from outside the cluster.
- Traffic is distributed to pods via ClusterIP.
- Suitable for development, testing, or environments without a cloud Load Balancer.

Example

Service Type : NodePort
Port (targetPort) : 80
NodePort : 30080
Access URL : http://203.0.113.10:30080

Port Range
30000 - 32767
(Default)

```
# 6_nodeportip_svc.yaml
```

```
---
apiVersion: v1
kind: Service
metadata:
  name: fb-service
  labels:
    app: fb-service
    env: prod
spec:
  type: NodePort #we use nodeport when we want to access the service outside the
cluster using node's IP and nodeport.
  selector:
    app: fb-app
    env: prod
  ports:
    - protocol: TCP
      port: 80 # This is the port to access the service, which is a external port.
      targetPort: 8080 # This is the port on which the application is running inside
the container.
      nodePort: 30080 # This is the port on which we can access the service outside the
cluster using node's IP and nodeport. It should be in the range of 30000-32767.
```

Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/3_June (main)

```
$ kubectl apply -f 6_nodeportip_svc.yaml
service/fb-service configured
```

Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/3_June (main)

\$ kubectl get all

NAME	READY	STATUS	RESTARTS	AGE
pod/fb-deployment-58d8bd65b9-n5bzg	1/1	Running	0	16m
pod/fb-deployment-58d8bd65b9-rdrj6	1/1	Running	0	16m
pod/fb-deployment-58d8bd65b9-znrsj	1/1	Running	0	16m

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
service/fb-service	NodePort	10.100.59.42	<none>	80:30080/TCP
18m				
service/kubernetes	ClusterIP	10.100.0.1	<none>	443/TCP
77m				

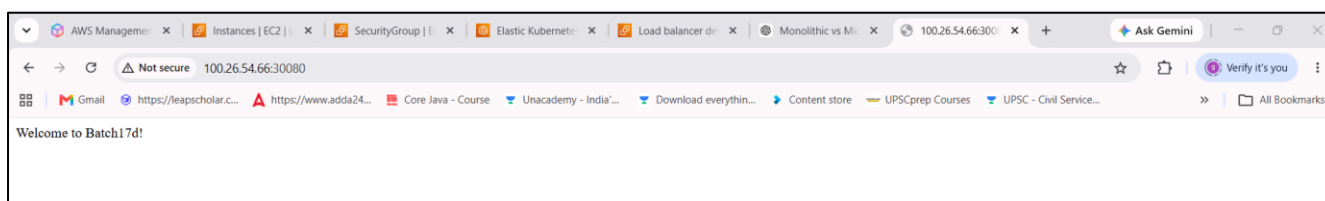
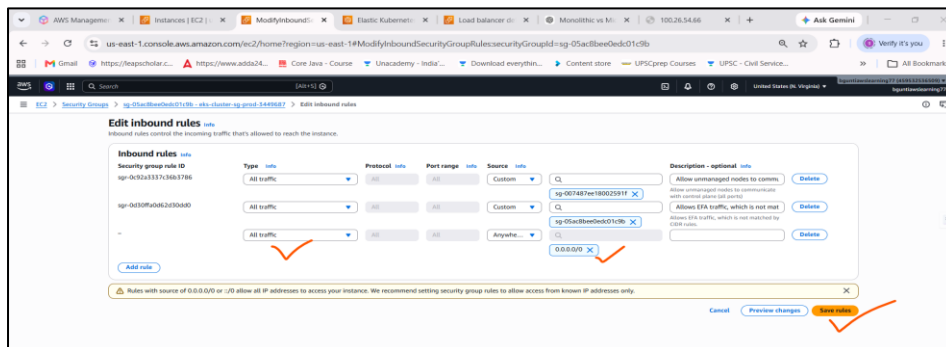
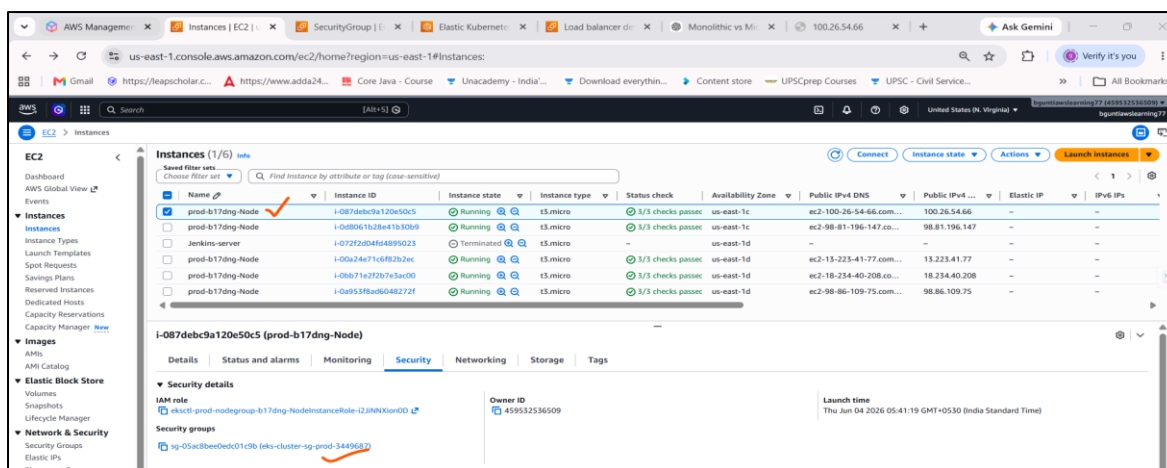
NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/fb-deployment	3/3	3	3	16m

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/fb-deployment-58d8bd65b9	3	3	3	16m

Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/3_June (main)

\$ kubectl get svc

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
fb-service	NodePort	10.100.59.42	<none>	80:30080/TCP	18m
fb-service	LoadBalancer	10.100.59.42	ab66a2684e4df455cbdd571b4d131320-		
1186501111.us-east-1.elb.amazonaws.com			80:30151/TCP	2m31s	
kubernetes	ClusterIP	10.100.0.1	<none>	443/TCP	78m



Namespaces

Interview Answer (Short)

Namespaces in Kubernetes are used to logically separate resources within a cluster. They help with environment isolation (dev, test, prod), access control, resource quotas, and better organization of applications while sharing the same cluster infrastructure.

In Kubernetes, Namespaces are used to logically separate resources within the same cluster. They help organize, secure, and manage applications efficiently.

Why Do We Use Namespaces?

1. Resource Isolation

Different teams or applications can use the same cluster without interfering with each other.

Example:

Namespace: dev

- └─ nginx-pod
- └─ frontend-service

Namespace: prod

- └─ nginx-pod
- └─ frontend-service

Here, both namespaces can have resources with the same names.

2. Environment Separation

You can create separate environments in a single cluster.

dev
testing
staging
production

This avoids the cost of maintaining multiple clusters.

3. Access Control (RBAC)

Permissions can be assigned namespace-wise.

Example:

Developers → Access only dev
QA Team → Access only testing
Admins → Access all namespaces

4. Resource Quotas

Limit CPU, Memory, and Storage usage per namespace.

Example:

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: dev-quota
  namespace: dev
spec:
  hard:
    requests.cpu: "4"
    requests.memory: 8Gi
```

This prevents one team from consuming all cluster resources.

5. Better Organization

Instead of having hundreds of Pods and Services in one place:

Without Namespace:

frontend
backend

redis
mysql
jenkins
prometheus
grafana

...

With Namespace:

dev

└─ frontend

└─ backend

prod

└─ frontend

└─ backend

monitoring

└─ prometheus

└─ grafana

cicd

└─ jenkins

Much easier to manage.

Real-Time Example

Suppose your company has:

Kubernetes Cluster

|

└─ Namespace: dev

|

└─ frontend

|

└─ backend

|

└─ mysql

|

└─ Namespace: prod

|

└─ frontend

|

└─ backend

|

└─ mysql

|

└─ Namespace: monitoring

|

└─ prometheus

|

└─ grafana

|

└─ Namespace: cicd

|

└─ jenkins

All applications run in the same cluster but remain logically separated.

Why we use Namespaces in Kubernetes?

Namespaces help us organize, isolate and manage resources in a Kubernetes cluster.

Key Reasons



1. Resource Isolation

Different teams or applications can use the same cluster without interfering with each other.



2. Environment Separation

Create separate environments (dev, test, staging, prod) in the same cluster.

dev test staging prod



3. Access Control (RBAC)

Permissions can be assigned namespace-wise.

Developers → dev
QA Team → test
Admins → all namespaces



4. Resource Quotas

Limit CPU, Memory and Storage usage per namespace.

Example: dev namespace
CPU: 4 cores
Memory: 8Gi



5. Better Organization

Keeps related resources grouped together and easy to manage.

Example: One Cluster with Multiple Namespaces



Kubernetes Cluster

Namespace: dev

frontend
backend
mysql

Namespace: test

frontend
backend
mysql

Namespace: prod

frontend
backend
mysql

Namespace: monitoring

prometheus
grafana

Namespace: cicd

jenkins

All namespaces share the same cluster resources but remain logically separated.

Common Namespace Commands

Create Namespace	<code>kubectl create namespace dev</code>
List Namespaces	<code>kubectl get ns</code>
Create Pod in Namespace	<code>kubectl apply -f pod.yaml -n dev</code>
View Resources in Namespace	<code>kubectl get pods -n dev</code>

Default Namespaces in Kubernetes

default	kube-system	kube-public	kube-node-lease
Resources go here if no namespace is specified.	Kubernetes system components.	Publicly accessible resources.	Node heartbeat information.

In Short

Namespaces help in isolation, security, resource management and organization – all within the same Kubernetes cluster.

```
Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/3_June (main)
```

```
$ cd ..
```

```
Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice (main)
```

```
$ cd 1_june/
```

```
Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/1_june (main)
```

```
$ ll
```

```
total 5
```

```
-rw-r--r-- 1 Bhavana 197121 342 Jun  2 11:46 1_pod.yaml
-rw-r--r-- 1 Bhavana 197121 455 Jun  2 11:59 2_pod.yaml
-rw-r--r-- 1 Bhavana 197121 343 Jun  2 12:34 3_multicont_pod.yaml
-rw-r--r-- 1 Bhavana 197121 612 Jun  2 13:04 4_rs.yaml
-rw-r--r-- 1 Bhavana 197121 218 Jun  2 13:45 5_pod.yaml
```

```
Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/1_june (main)
```

```
$ kubectl apply -f 1_pod.yaml
pod/fb-pod created
```

```
Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/1_june (main)
```

```
$ kubectl get all
```

NAME	READY	STATUS	RESTARTS	AGE
pod/fb-pod	1/1	Running	0	13s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/kubernetes	ClusterIP	10.100.0.1	<none>	443/TCP	5m8s

```
Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/1_june (main)
```

```
$ kubectl apply -f 1_pod.yaml
pod/fb-pod unchanged
```



```

Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-
Devops_MC_B17D/Kubernetes/practice/1_june (main)
$ kubectl describe pod fb-pod
Name: fb-pod
Namespace: default
Priority: 0
Service Account: default
Node: ip-192-168-17-129.ec2.internal/192.168.17.129
Start Time: Thu, 04 Jun 2026 07:27:02 +0530
Labels: app=fb-app
        env=prod
Annotations: <none>
Status: Running
IP: 192.168.10.111
IPs:
  IP: 192.168.10.111
Containers:
  fb-container:
    Container ID:
    containerd://c180268d2862ebe1111dc14fb28e1f6b761d1e1640803f1a2217e1e13666388f
    Image: tomcat
    Image ID:
    docker.io/library/tomcat@sha256:f8d2287df1bb2bf9b75a6acfc6d25a75b3a9044429906f
    f04e01533cfbafa028
    Port: 8080/TCP
    Host Port: 0/TCP

Events:
  Type        Reason          Age   From                      Message
  ----        -
  Normal      Scheduled       3m7s  default-scheduler        Successfully assigned
  default/fb-pod to ip-192-168-17-129.ec2.internal
  Normal      Pulling         3m7s  kubelet                  spec.containers{fb-container}:
  Pulling image "tomcat"
  Normal      Pulled          3m1s  kubelet                  spec.containers{fb-container}:
  Successfully pulled image "tomcat" in 6.01s (6.01s including waiting). Image
  size: 154491880 bytes.
  Normal      Created         3m1s  kubelet                  spec.containers{fb-container}:
  Created container: fb-container
  Normal      Started         3m    kubelet                  spec.containers{fb-container}:
  Started container fb-container

```

```

Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-
Devops_MC_B17D/Kubernetes/practice/1_june (main)
$ kubectl create namespace dev
namespace/dev created

```

```

Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-
Devops_MC_B17D/Kubernetes/practice/1_june (main)
$ kubectl create namespace qa
namespace/qa created

```

```

Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-
Devops_MC_B17D/Kubernetes/practice/1_june (main)
$ kubectl create namespace prod
namespace/prod created

```

```

Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-
Devops_MC_B17D/Kubernetes/practice/1_june (main)
$ kubectl get ns
NAME                STATUS    AGE
default             Active    115m
dev                 Active    34s
kube-node-lease     Active    115m
kube-public         Active    115m
kube-system         Active    115m
prod                Active    15s
qa                  Active    22s

```

```
Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/1_june (main)
$ kubectl apply -f 1_pod.yaml -n qa
pod/fb-pod created
```

```
Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/1_june (main)
$ kubectl apply -f 1_pod.yaml -n dev
pod/fb-pod created
```

```
Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/1_june (main)
$ kubectl get po -n qa
NAME      READY   STATUS    RESTARTS   AGE
fb-pod    1/1     Running   0           37s
```

```
Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/1_june (main)
$ kubectl get po -n dev
NAME      READY   STATUS    RESTARTS   AGE
fb-pod    1/1     Running   0           26s
```

```
Bhavana@LAPTOP-E0A1LIK MINGW64 ~/Downloads/Bhavana work/AWS Devops/AWS-Devops_MC_B17D/Kubernetes/practice/1_june (main)
$ kubectl get po --all-namespaces
NAMESPACE   NAME                                     READY   STATUS    RESTARTS   AGE
default     fb-pod                                 1/1     Running   0           6m16s
dev          fb-pod                                 1/1     Running   0           63s
kube-system  aws-node-6r6l9                         2/2     Running   0           111m
kube-system  aws-node-ggcmz                         2/2     Running   0           111m
kube-system  aws-node-kdnr6                         2/2     Running   0           111m
kube-system  aws-node-v2fjh                         2/2     Running   0           111m
kube-system  aws-node-zv15f                         2/2     Running   0           111m
kube-system  coredns-566b9b9d-8wdkw                 1/1     Running   0           114m
kube-system  coredns-566b9b9d-dphx1                 1/1     Running   0           114m
kube-system  kube-proxy-6nxzw                       1/1     Running   0           111m
kube-system  kube-proxy-mmhxq                       1/1     Running   0           111m
kube-system  kube-proxy-prt69                       1/1     Running   0           111m
kube-system  kube-proxy-rqhg5                       1/1     Running   0           111m
kube-system  kube-proxy-x5bbv                       1/1     Running   0           111m
kube-system  metrics-server-694dffc97b-q89rw        1/1     Running   0           109m
kube-system  metrics-server-694dffc97b-xtqjr        1/1     Running   0           109m
qa           fb-pod
```

```

---
apiVersion: v1
kind: Pod
metadata:
  name: fb-pod #pod name
  labels:
    app: fb-app # this is the label of the pod, we can use this label to select the
pod.
  namespace: prod

spec:
  containers:
  - name: fb-container #container name used in the pod
    image: tomcat
    ports:
      - containerPort: 8080 # tomcat server port.

```

kubectl apply -f pod.yaml

Verify

Check namespaces:

kubectl get ns

Check pods in prod namespace:

kubectl get pods -n dev

Expected output:

NAME	READY	STATUS	RESTARTS	AGE
nginx-pod	1/1	Running	0	1m